

dati-basati

Draw Entity-Relations diagrams.

<https://github.com/victuarvi/dati-basati>

Version 0.1.1

Manual

1. Import	2
2. Basic usage	3
2.1. Entities	3
2.1.1. Entity with attributes . . .	3
2.1.2. Subentities	5
2.1.3. Attributes positioning . .	8
2.2. Relations	11
3. Formatting & customization	14
3.1. Individual styling overrides . . .	17
3.2. Themes showcase	18
4. Advanced usage	24
4.1. Weak entities	24
4.2. Unorthodox relation position .	26
4.3. Exceptions	26
4.4. Touying integration	27

Documentation

5. User functions	28
5.1 dati-basati	28
5.2 er-diagram	29
5.3 entity	30
5.4 subentities	31
5.5 relation	32

1. Import

Depending on your needs there are different types of imports:

- Import *all* the functions (recommended¹):

typ

```
1 #import "@preview/dati-basati:0.1.1": *
```

- Import *only* the functions you want to use:

typ

```
1 #import "@preview/dati-basati:0.1.1": er-diagram, entity, relation
```

- Import and (optionally) rename:

typ

```
1 #import "@preview/dati-basati:0.1.1" as renamed
```

but your code would be more verbose, like in the following example:

typ

```
1 #renamed.er-diagram({  
2   renamed.entity(  
3     (0, 0),  
4     name: "entity0",  
5   )  
6 })
```

The package is designed to be as straightforward and intuitive as possible, so you should be able to just write diagrams without even needing to read the manual – or at least that was the design goal.

¹Since it's unlikely there will be conflicts with identically named functions.

2. Basic usage

2.1. Entities

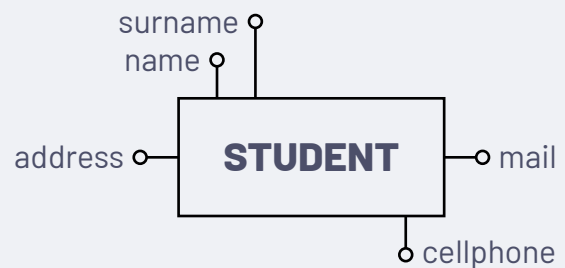
A basic entity would appear like this:

```
#er-diagram({  
  entity(  
    // coordinates  
    (0, 0),  
    // the internal reference  
    name: "entity",  
    // the content label  
    label: "Nice Name"  
  )  
})
```



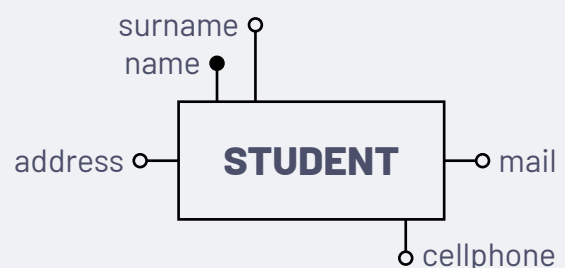
2.1.1. Entity with attributes

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "student",  
    label: "student",  
    attributes: (  
      north: ("name", "surname"),  
      east: ("mail",),  
      west: ("address",),  
      south: ("cellphone",),  
    ),  
  )  
})
```



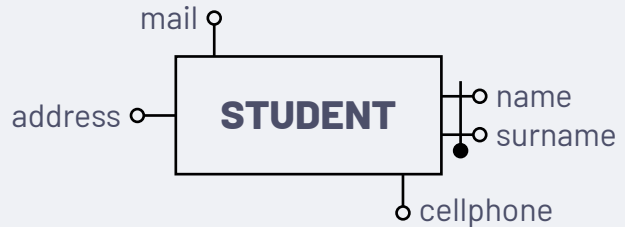
Primary key Adding a single primary key:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "student",  
    label: "student",  
    attributes: (  
      north: ("name", "surname"),  
      east: ("mail",),  
      west: ("address",),  
      south: ("cellphone",),  
    ),  
    primary-key: "name"  
  )  
})
```



Composite primary key Adding a composite primary key:

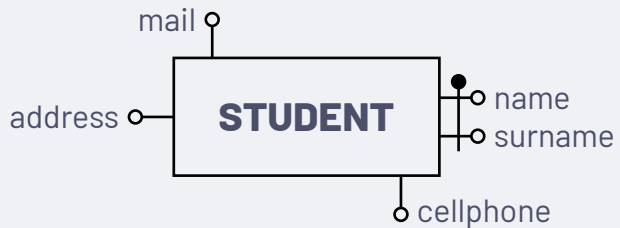
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("mail",),
      east: ("name", "surname"),
      west: ("address",),
      south: ("cellphone",),
    ),
    primary-key: ("name", "surname")
  )
})
```



i Info

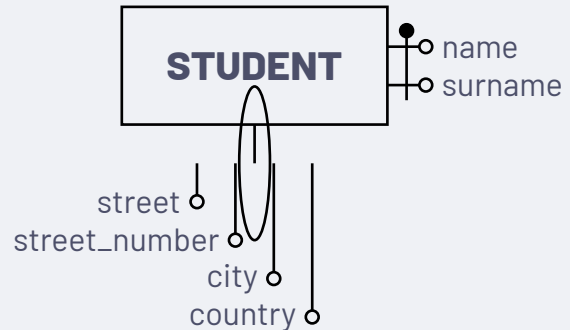
The direction of the line will depend on the exact order you give the attributes:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("mail",),
      east: ("name", "surname"),
      west: ("address",),
      south: ("cellphone",),
    ),
    primary-key: ("name", "surname").rev()
  )
})
```



Composite attributes You might want to group some attributes together:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      east: ("name", "surname"),
      south: (
        // instead of a string, the comp.
        // attr. is an array
        ("street", "street_number",
        "city", "country"),
      ),
    ),
    primary-key: ("name", "surname").rev()
  )
})
```

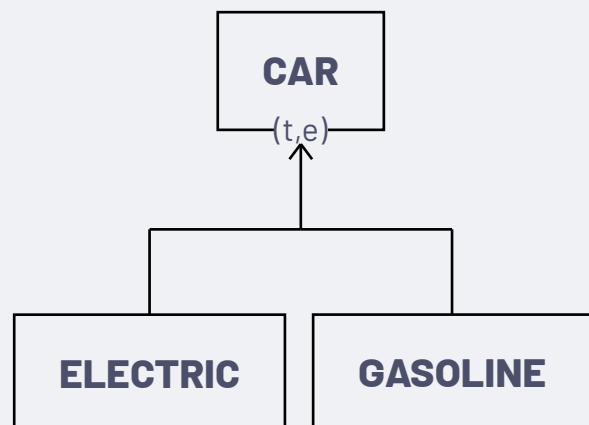


i Info

The composite attributes is designed to be *the only attribute* on its side, meaning that other attributes on the same side will NOT be drawn.

2.1.2. Subentities

```
#er-diagram({
  entity(
    (0, 4),
    name: "A",
    label: "car",
  )
  entity(
    (-2, 0),
    name: "B",
    label: "electric",
  )
  entity(
    (2, 0),
    name: "C",
    label: "gasoline",
  )
  subentities(
    entity: "A",
    subentities: ("B", "C"),
  )
})
```



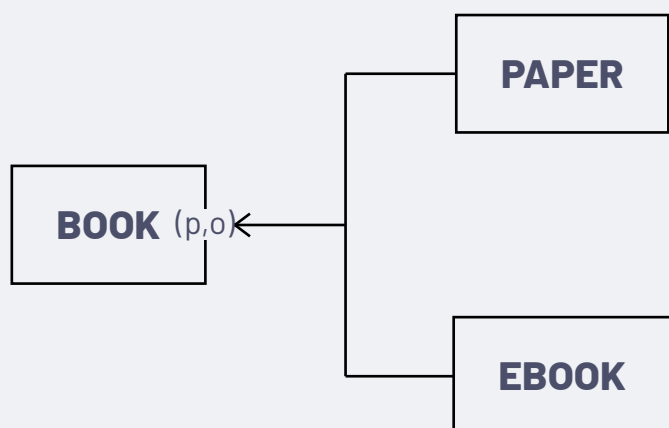
⚠ Warning

You can't connect a subentity to an entity if that entity has not been drawn yet: typ

```
1 // this will NOT work
2 #er-diagram({
3   subentities(
4     entity: "A",
5     subentities: ("B", "C"),
6   )
7   entity(
8     (0, 0),
9     name: "A",
10    label: "entity1",
11  )
12  entity(
13    (-2, 4),
14    name: "B",
15    label: "entity2",
16  )
17 })
```

Although subentities generally tend to be below their superentity, you can place them wherever you like best:

```
#er-diagram({
  entity(
    (0, 0),
    name: "A",
    label: "book",
  )
  entity(
    (6, 2),
    name: "B",
    label: "paper",
  )
  entity(
    (6, -2),
    name: "C",
    label: "ebook",
  )
  subentities(
    hierarchy: "(p,o)",
    entity: "A",
    subentities: ("B", "C"),
  )
})
```

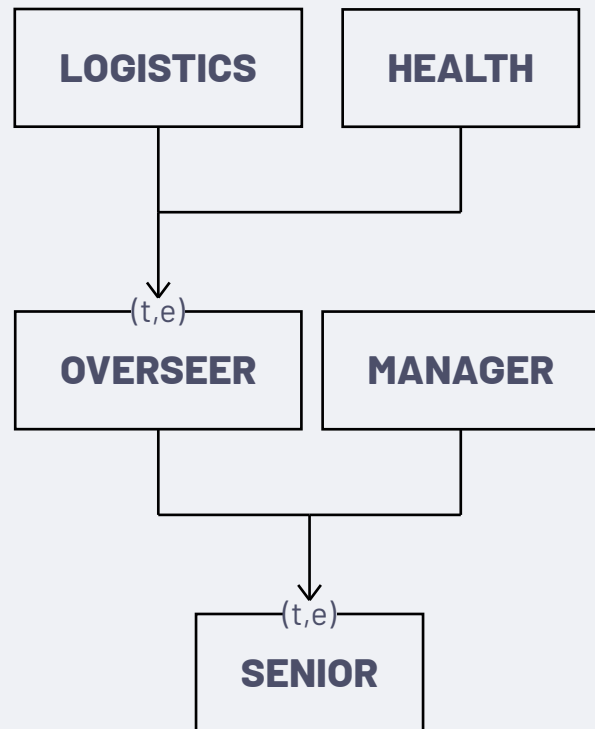


A more complex layout:

```

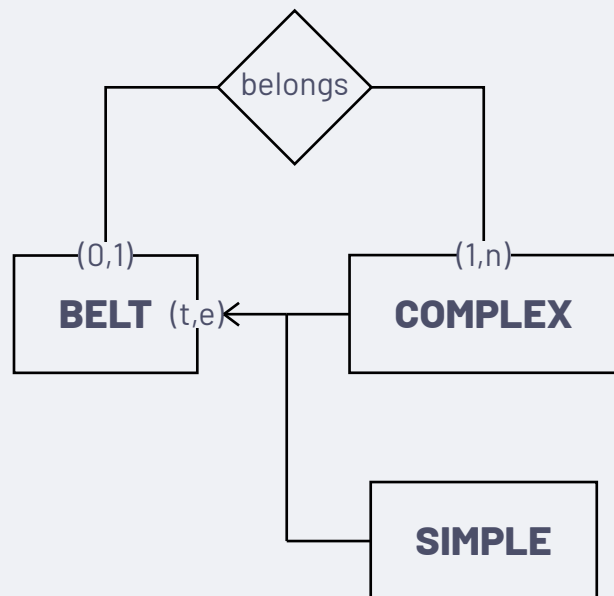
#er-diagram({
  entity(
    (0, 0),
    name: "A",
    label: "senior",
  )
  entity(
    (-2, 4),
    name: "B",
    label: "overseer",
  )
  entity(
    (2, 4),
    name: "C",
    label: "manager",
  )
  entity(
    (2, 8),
    name: "D",
    label: "health",
  )
  entity(
    (-2, 8),
    name: "E",
    label: "logistics",
  )
  subentities(
    entity: "A",
    subentities: ("B", "C"),
  )
  subentities(
    entity: "B",
    subentities: ("D", "E"),
  )
})

```



If you need a relation between an entity and one of its subentities you can do so via the intersect argument:

```
#er-diagram({
  entity(
    (0, 0),
    label: "belt",
    name: "belt",
  )
  entity(
    (5, 0),
    label: "complex",
    name: "complex",
  )
  entity(
    (5, -3),
    label: "simple",
    name: "simple",
  )
  relation(
    coordinates: (2.5, 3),
    intersect: true,
    entities: ("belt", "complex"),
    label: "belongs",
    name: "belt-complex",
    cardinality: ("(0,1)", "(1,n)"),
  )
  subtentities(
    hierarchy: "(t,e)",
    entity: "belt",
    subtentities: ("complex", "simple"),
  )
})
```



2.1.3. Attributes positioning

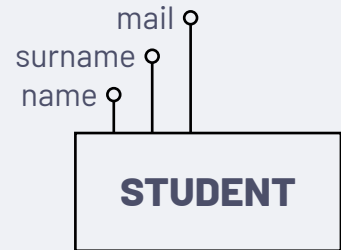
Depending on what you are trying to achieve, the default placement could be wrong – you can modify it with `attributes-position`:

typ

```
1 attributes-position: (
2   side: (
3     alignment: left | center | right,
4     dir: "ltr" | "rtl",           // left-to-right, right-to-left
5     start: "from-short" | "from-long" // from shortest to longest
6   ),
7 )
```

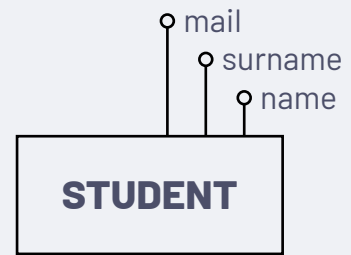
Default:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
  )
})
```



Mirrored:

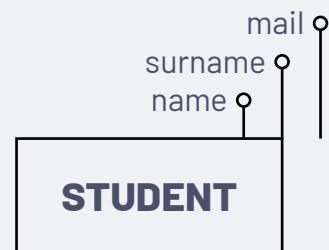
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (alignment: right, dir: "rtl")
    ),
  )
})
```



⚠ Warning

You will find out *some combinations* will lead to visual errors:

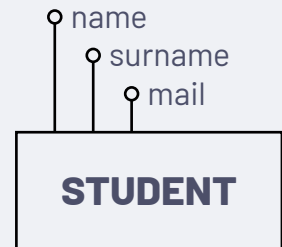
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (alignment: right)
    )
  )
})
```



In the future this might be patched.

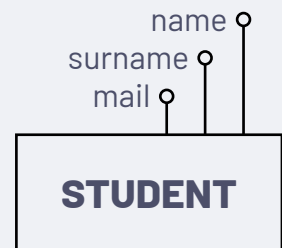
From high to low:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "student",  
    label: "student",  
    attributes: (  
      north: ("name", "surname", "mail"),  
    ),  
    attributes-position: (  
      north: (start: "from-long")  
    )  
  )  
})
```



Mirrored:

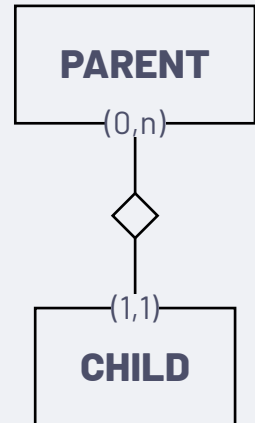
```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "student",  
    label: "student",  
    attributes: (  
      north: ("name", "surname", "mail"),  
    ),  
    attributes-position: (  
      north: (  
        start: "from-long",  
        alignment: right,  
        dir: "rtl"  
      )  
    )  
  )  
})
```



2.2. Relations

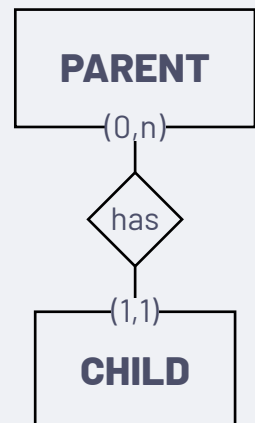
Ideally the relation should be a straight line connecting two entities:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "child",  
    label: "child",  
  )  
  entity(  
    (0, 4),  
    name: "parent",  
    label: "parent",  
  )  
  relation(  
    entities: ("parent", "child"),  
    cardinality: ("(0,n)", "(1,1)"),  
  )  
})
```



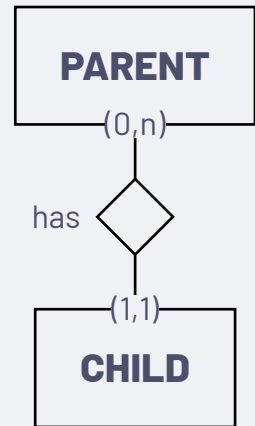
With label:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "child",  
    label: "child",  
  )  
  entity(  
    (0, 4),  
    name: "parent",  
    label: "parent",  
  )  
  relation(  
    label: "has",  
    entities: ("parent", "child"),  
    cardinality: ("(0,n)", "(1,1)"),  
  )  
})
```



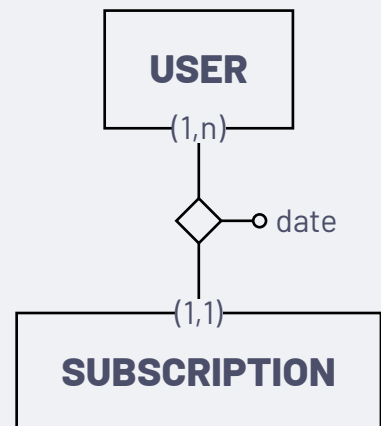
With label outside:

```
#er-diagram({
  entity(
    (0, 0),
    name: "child",
    label: "child",
  )
  entity(
    (0, 4),
    name: "parent",
    label: "parent",
  )
  relation(
    label: ("has", "east"),
    entities: ("parent", "child"),
    cardinality: ("(0,n)", "(1,1)"),
  )
})
```



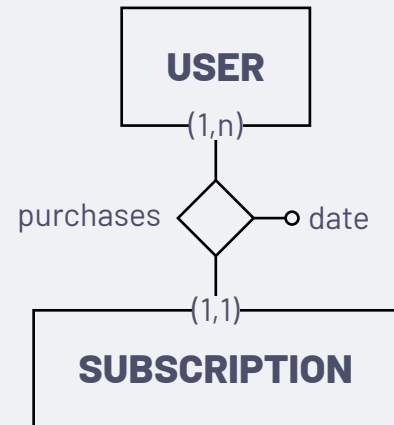
With an attribute (same API as entities, see [Section 2.1.1](#)):

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (0, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
    attributes: (east: ("date",)),
  )
})
```



Combined:

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (0, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    label: ("purchases", "east"),
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
    attributes: (east: ("date",)),
  )
})
```

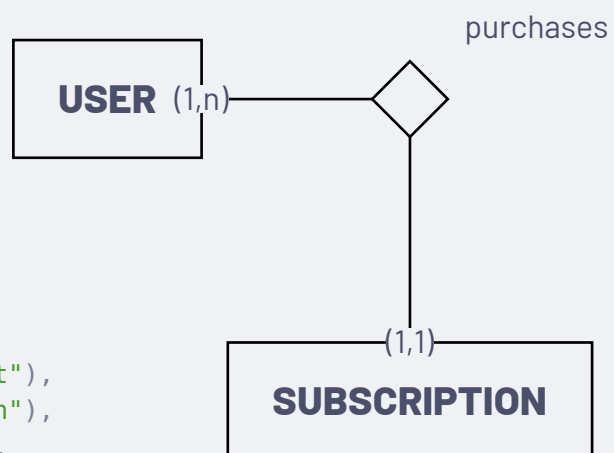


i Info

Relations **can't** have primary keys; also there is no such thing as a "weak relation" (see [Section 4.1](#)).

Manual placement² If you need to manually specify where to place a relation you can:

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (4, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    coordinates: (4,0),
    label: ("purchases", "south-west"),
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



²Since the relation attributes API is the same as entities, [Section 2.1.3](#) would apply as expected. Though remember the relations don't have... many anchors to draw from.

3. Formatting & customization

The package provides an handful of themes you can use by importing the corresponding module and applying the show rule:

typ

```
1 #import "@preview/dati-basati:0.1.1" as db
2 #show: db.with(..db.themes.at("theme-name"))
```

Currently the themes are:

- default
- tiramisu
- polimi
- tiw
- futurama
- vaporvawe
- snow
- C62-50
- gali
- ghibli
- C62-48

The theme schema is as follows:

typ

```
1 #let default-theme = (
2   fill: (
3     entities: none,
4     relations: auto, // auto inherits entities
5     primary-key: black,
6     weak-entity: auto, // auto inherits primary-key
7     cardinality: auto, // auto = page fill
8     hierarchy: auto, // auto inherits cardinality
9     composite-attributes: auto, // auto inherits entities
10  ),
11  stroke: (
12    entities: black,
13    relations: black, // auto inherits entities
14    lines: auto, // auto inherits entities
15    attributes: black,
16    composite-attributes: auto, // auto inherits attributes
17    primary-key: auto, // auto inherits attributes
18    weak-entity: auto, // auto inherits primary-key
19    cardinality: none,
20    hierarchy: auto, // auto inherits cardinality
21  ),
22  radius: (
23    entities: 0pt,
24    cardinality: 0pt,
25    hierarchy: auto, // auto inherits cardinality
26  ),
27  spacing: (
28    in-between: (x: 1.2em, y: 1.2em), // spacing between attributes
29    padding: 1.2em, // distance from the entity
30  ),
31  text: (
32    entities: l => {
33      set par(leading: 0.35em)
```

```

34     text(
35         top-edge: "bounds",
36         size: 1.5em,
37         weight: "bold",
38         smallcaps(lower(l)),
39     )
40 },
41 relations: l => l,
42 relations-outside: auto, // auto inherits relations
43 attributes: l => l,
44 cardinality: l => text(top-edge: "bounds", bottom-edge: "bounds", l),
45 hierarchy: auto,
46 ),
47 attributes-position: (
48     "north": (
49         alignment: left,
50         dir: "ltr",
51         start: "from-short",
52     ),
53     "east": (
54         alignment: center,
55         dir: "ltr",
56     ),
57     "south": (
58         alignment: right,
59         dir: "rtl",
60         start: "from-short",
61     ),
62     "west": (
63         alignment: center,
64         dir: "rtl",
65     ),
66 ),
67 misc: (
68     weak-entities-stroke: false, // double stroke weak entities
69     relations-intersection: "|-",
70 ),
71 )

```

Global theme By design, a theme is to be set globally:

typ

```

1 #import "@preview/dati-basati:0.1.1" as db
2 #show: db.dati-basati.with(theme: themes.at("theme-name"))

```

And it can be overridden:

typ

```

1 #import "@preview/dati-basati:0.1.1" as db
2 #show: db.dati-basati.with(theme: themes.at("theme-name"), entities:
  (fill: green))

```

You can also create your own theme:

typ

```
1 #import "@preview/dati-basati:0.1.1" as db
2 #import "@preview/catpuccin:1.0.1": *
3 #let palette = flavors.latte.colors
4 #let catpuccin-red-latte-theme = (
5   fill: (
6     cardinality: palette.flamingo.rgb,
7     entities: palette.red.rgb,
8     relations: palette.red.rgb,
9     attributes: palette.red.rgb,
10    primary-key: palette.maroon.rgb,
11    weak-entity: palette.red.rgb,
12  ),
13  stroke: (
14    cardinality: palette.surface0.rgb + 0.1pt,
15    entities: black,
16    relations: black,
17    attributes: palette.rosewater.rgb,
18    weak-entity: orange,
19  ),
20  text: (
21    entities: l => text(
22      fill: palette.surface0.rgb,
23      weight: 500,
24      size: 1.2em,
25      smallcaps(l),
26    ),
27    cardinality: l => text(
28      top-edge: "bounds",
29      bottom-edge: "bounds",
30      fill: palette.surface0.rgb,
31      l,
32    ),
33  ),
34  radius: (
35    entities: 6pt,
36    cardinality: 6pt,
37  ),
38 )
39 #show: db.dati-basati.with(theme: catpuccin-red-latte-theme)
```

See [example 14](#) to see how this theme looks.

There are plenty of examples with each and every theme – and corresponding source code – in [/examples](#).

Local setting You also set a theme in an individual diagram:

typ

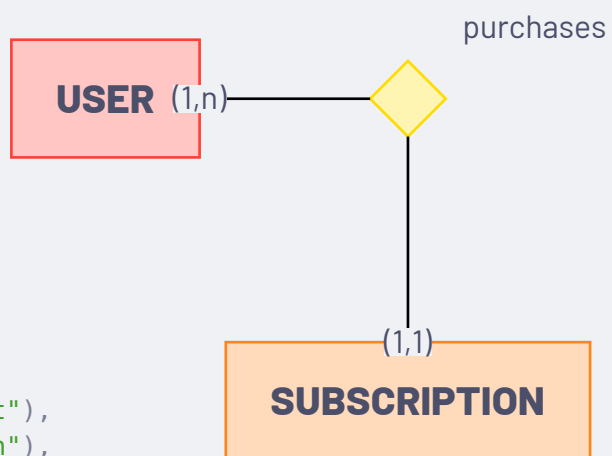
```
1 #import "@preview/dati-basati:0.1.1" as db
2 #show: db.dati-basati.with(theme: themes.at("global-theme"))
3
4 // this will use the local theme
5 #db.er-diagram(theme: themes.at("local-theme"), {...})
6
7 // this will use the global theme
8 #db.er-diagram({...})
```

The overrides still apply as expected.

3.1. Individual styling overrides

At the start of the document the user is able to declare the global styling arguments, however there can be the need to individually override elements:

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
    fill: red.lighten(70%),
    stroke: red,
  )
  entity(
    (4, -4),
    name: "subscription",
    label: "subscription",
    fill: orange.lighten(70%),
    stroke: orange,
  )
  relation(
    coordinates: (4,0),
    label: ("purchases", "south-west"),
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
    fill: yellow.lighten(70%),
    stroke: yellow
  )
})
```



3.2. Themes showcase

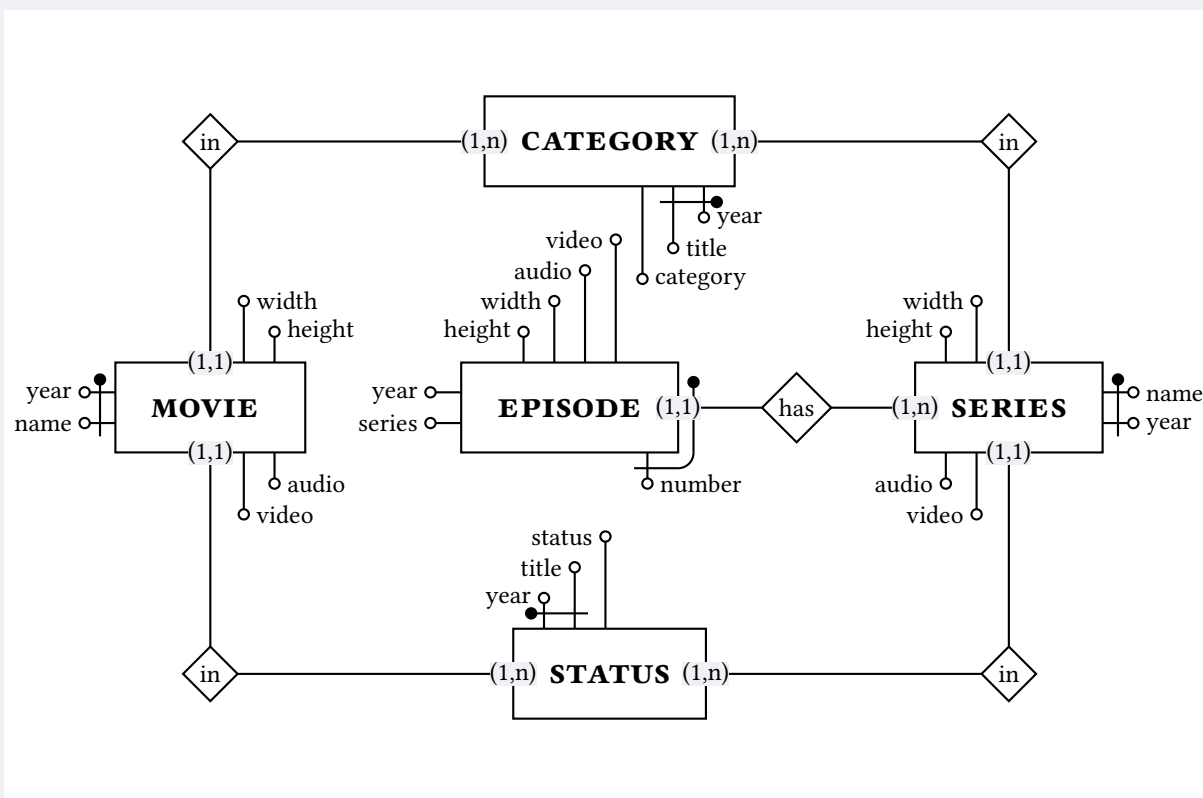


Figure 1: default theme (font: Libertinus Serif).

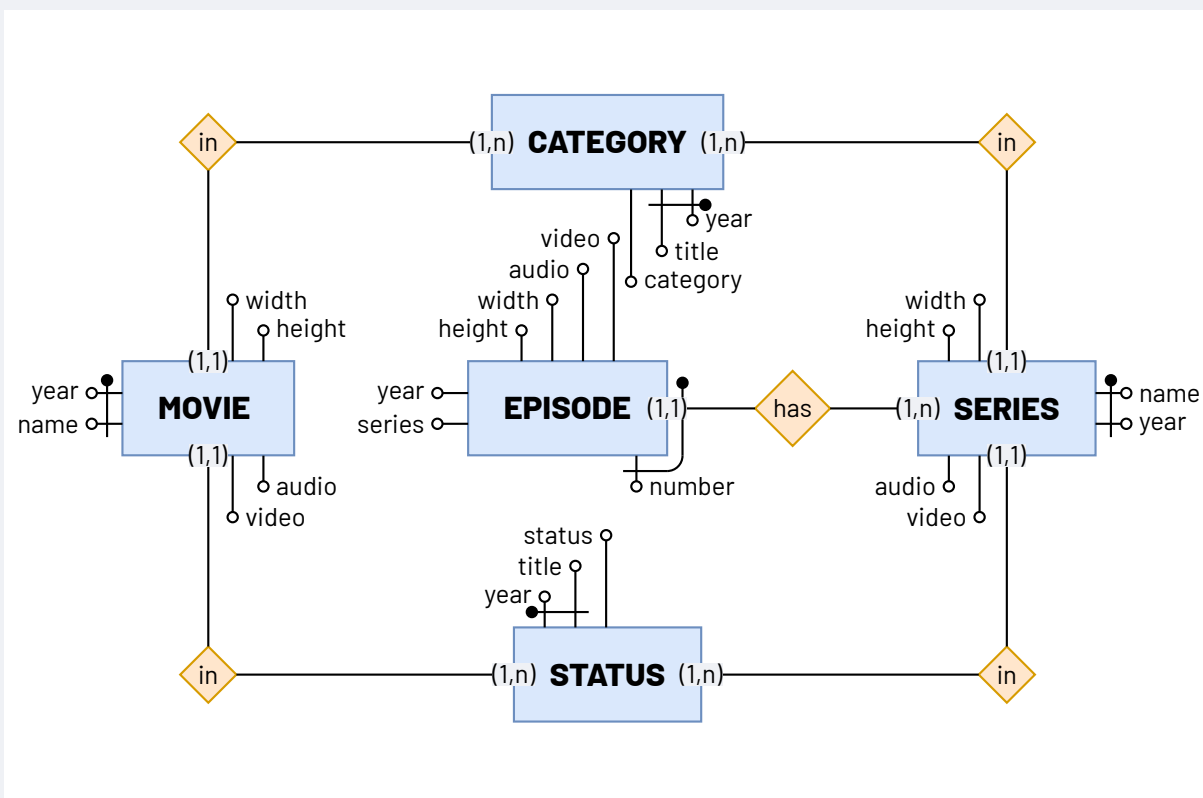


Figure 2: tiw theme (font: Barlow).

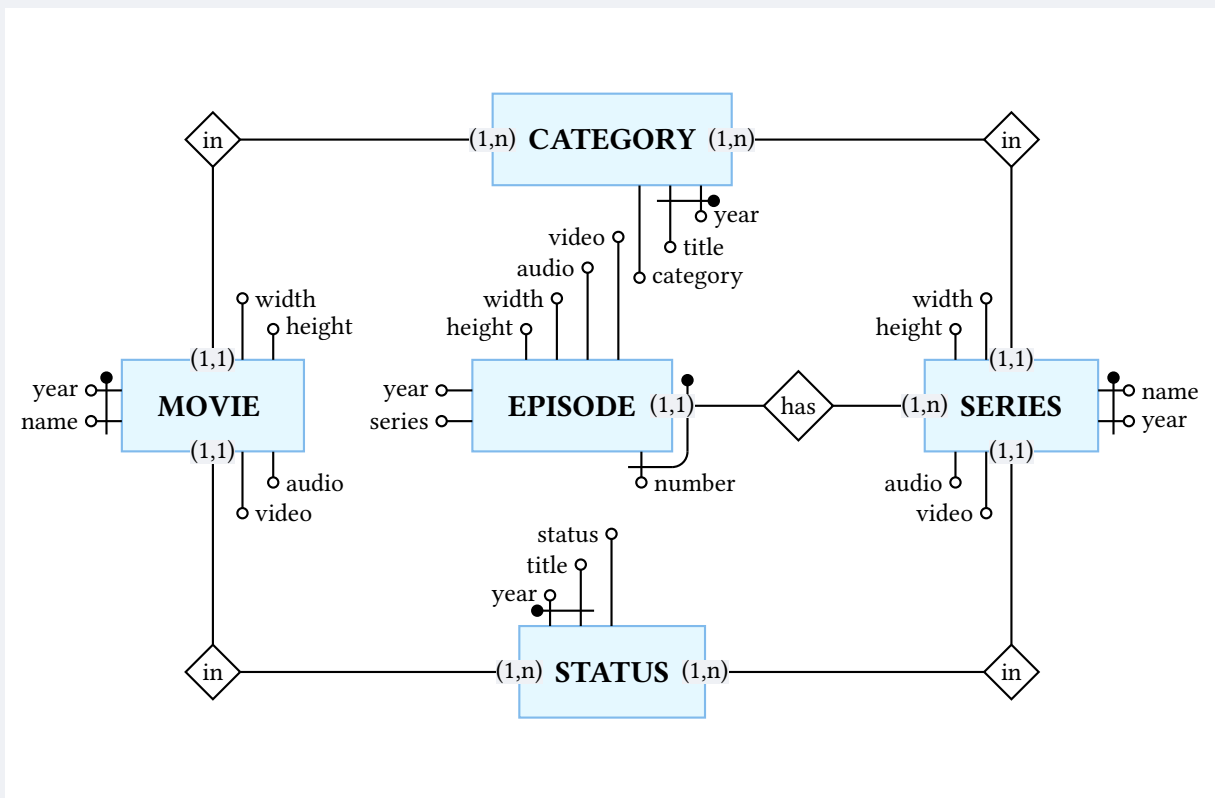


Figure 3: snow theme (font: Libertinus Serif).

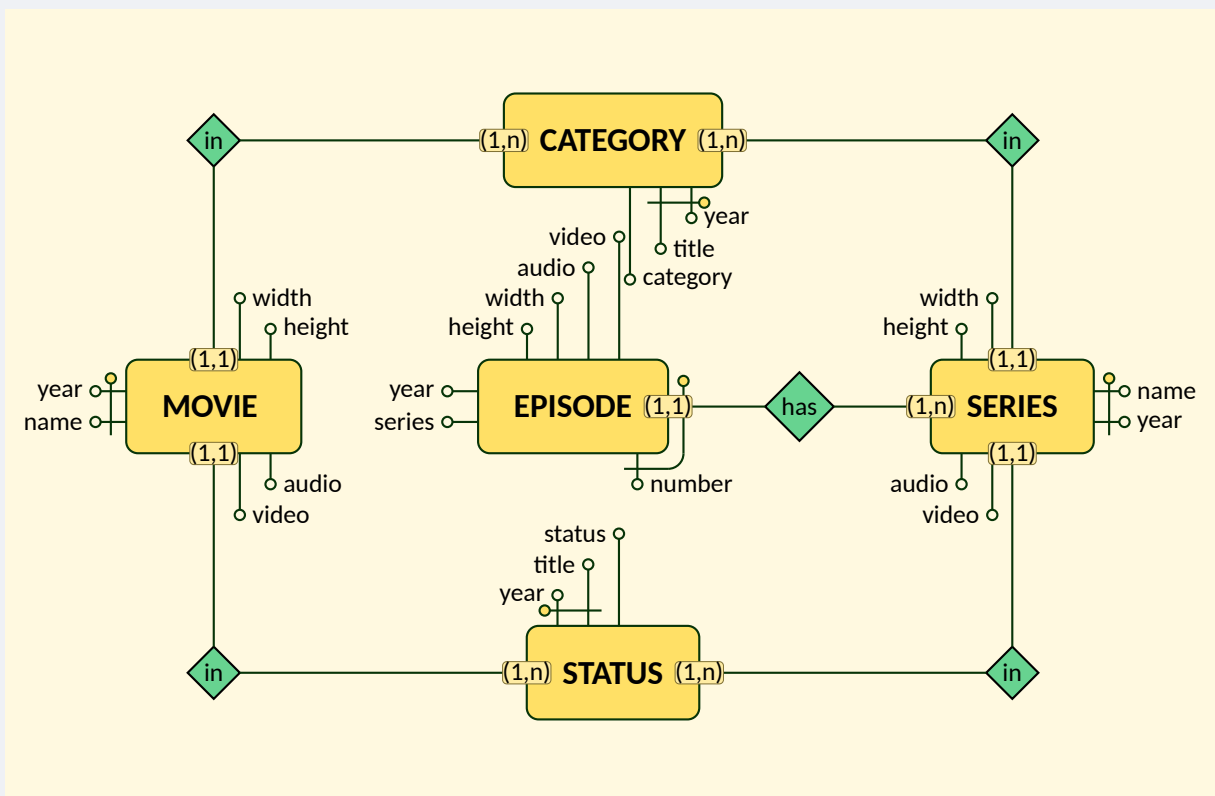


Figure 4: ghibli theme (font: Carlito).

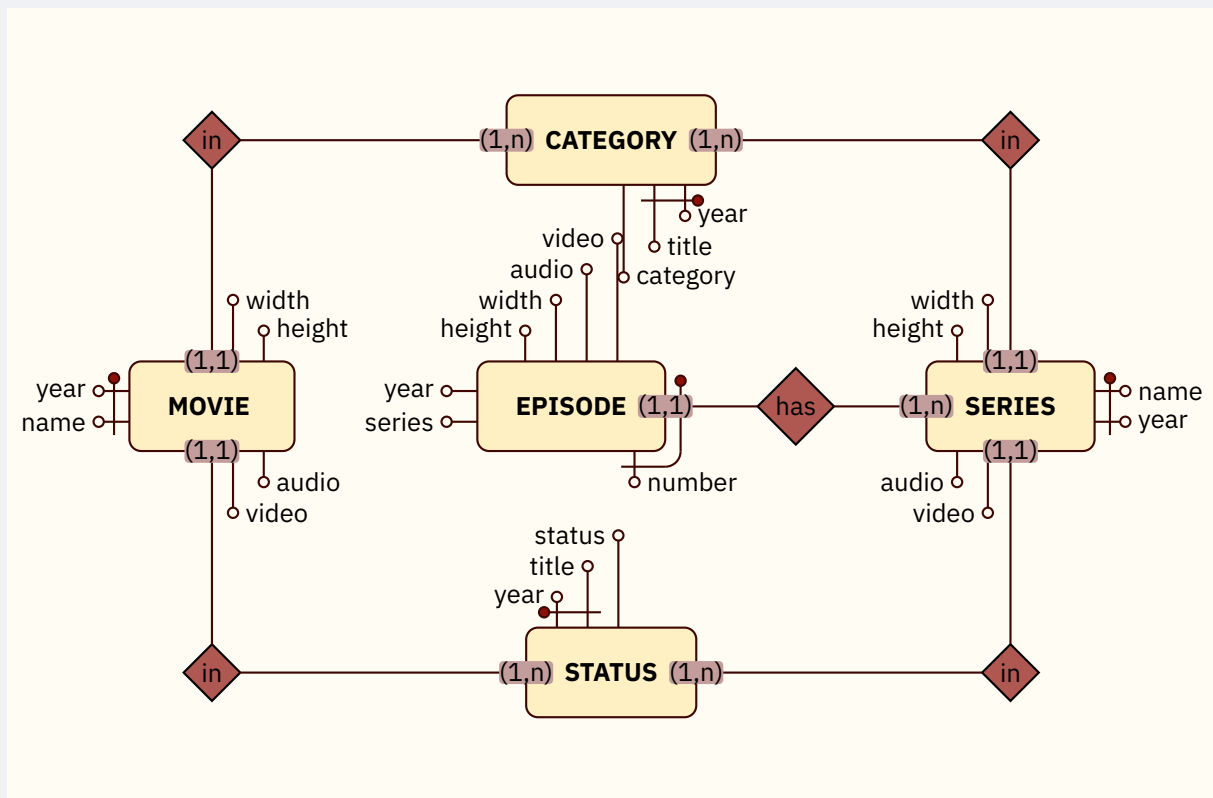


Figure 5: tiramisu theme (font: IBM Plex Sans).

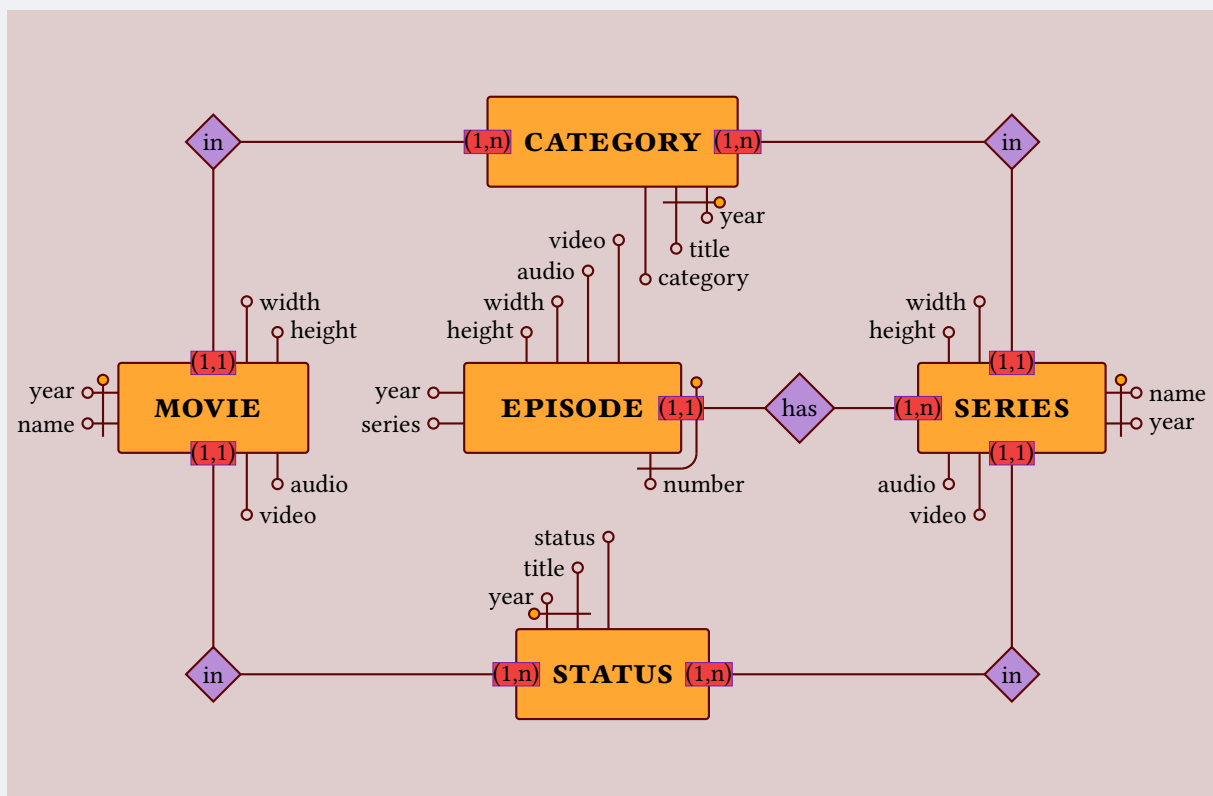


Figure 6: futurama theme (font: Libertinus Serif).

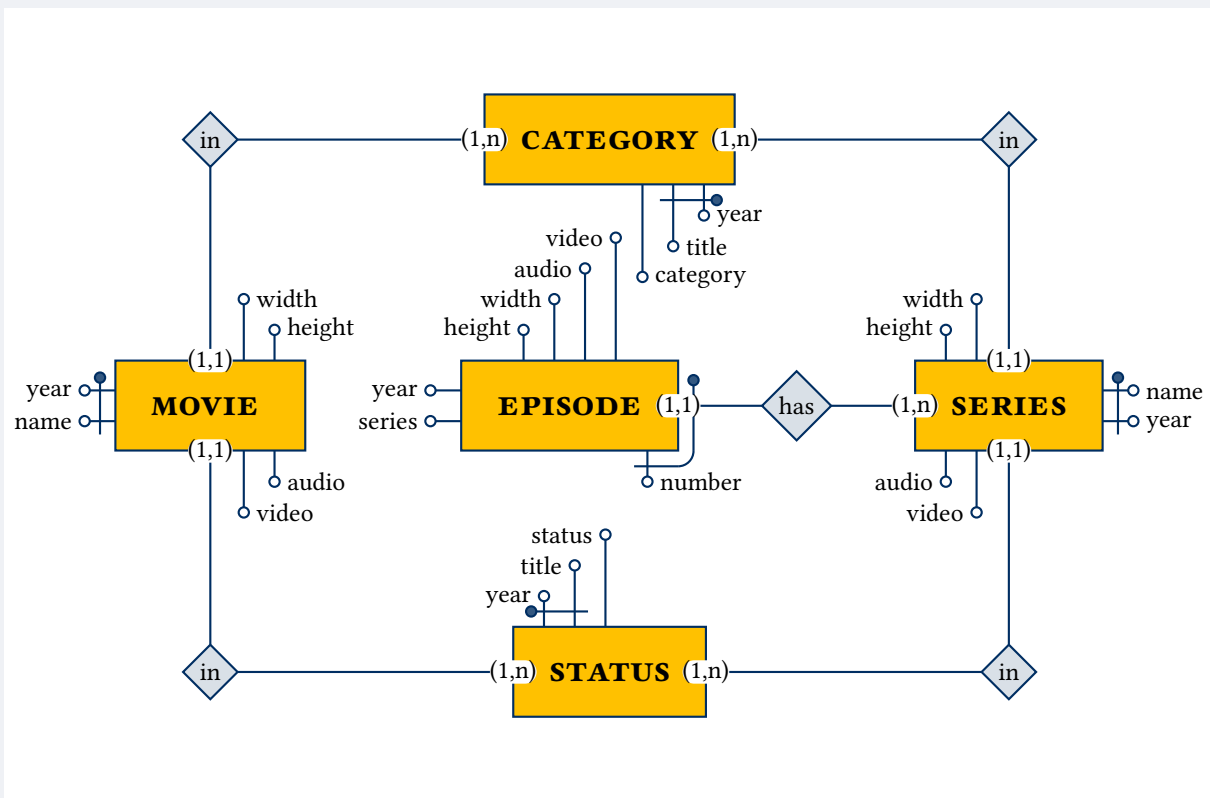


Figure 7: C62-50 theme (font: Libertinus Serif).

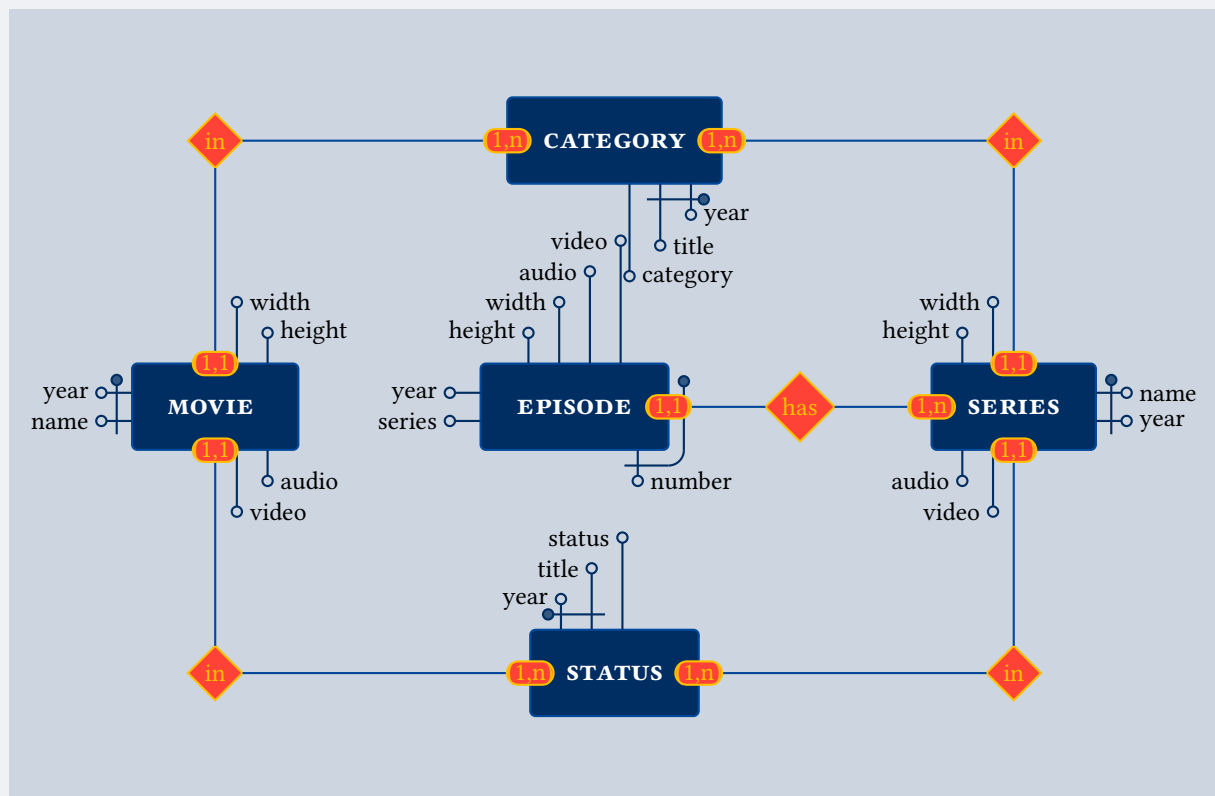


Figure 8: C62-48 theme (font: Libertinus Serif).

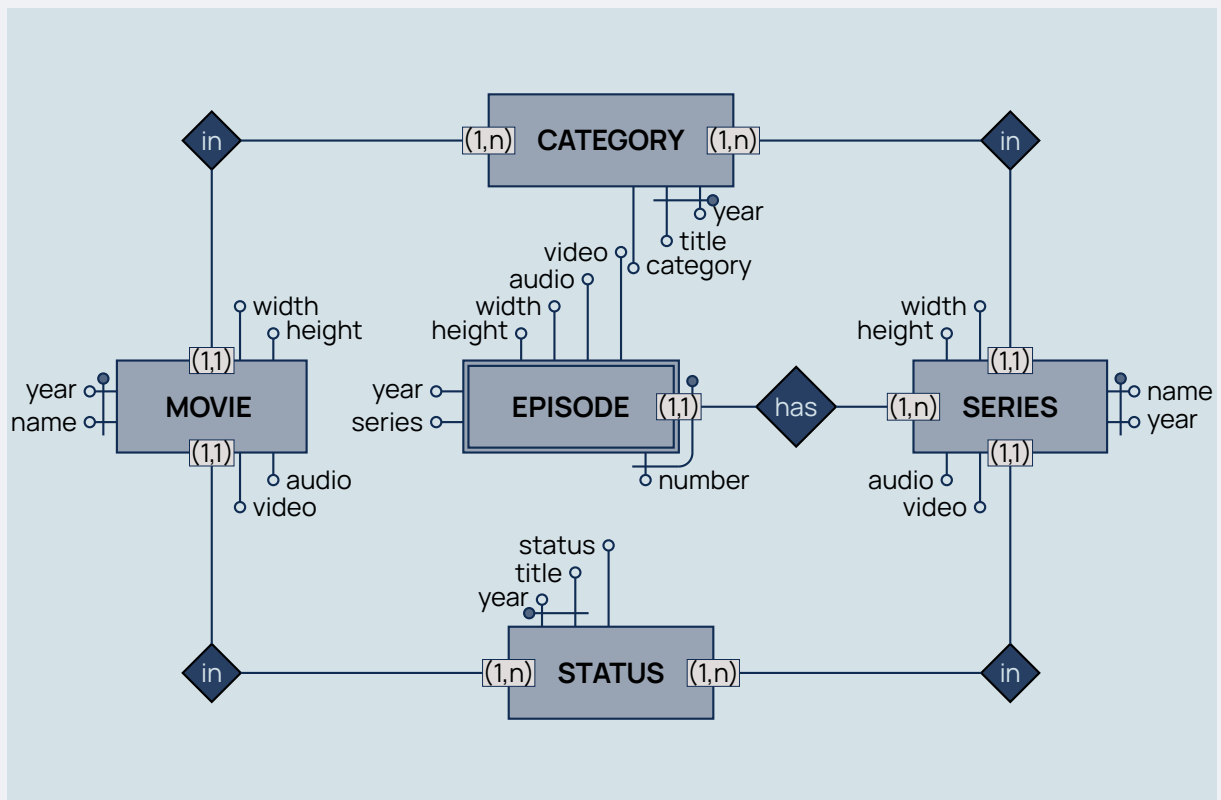


Figure 9: polimi theme (font: Manrope).

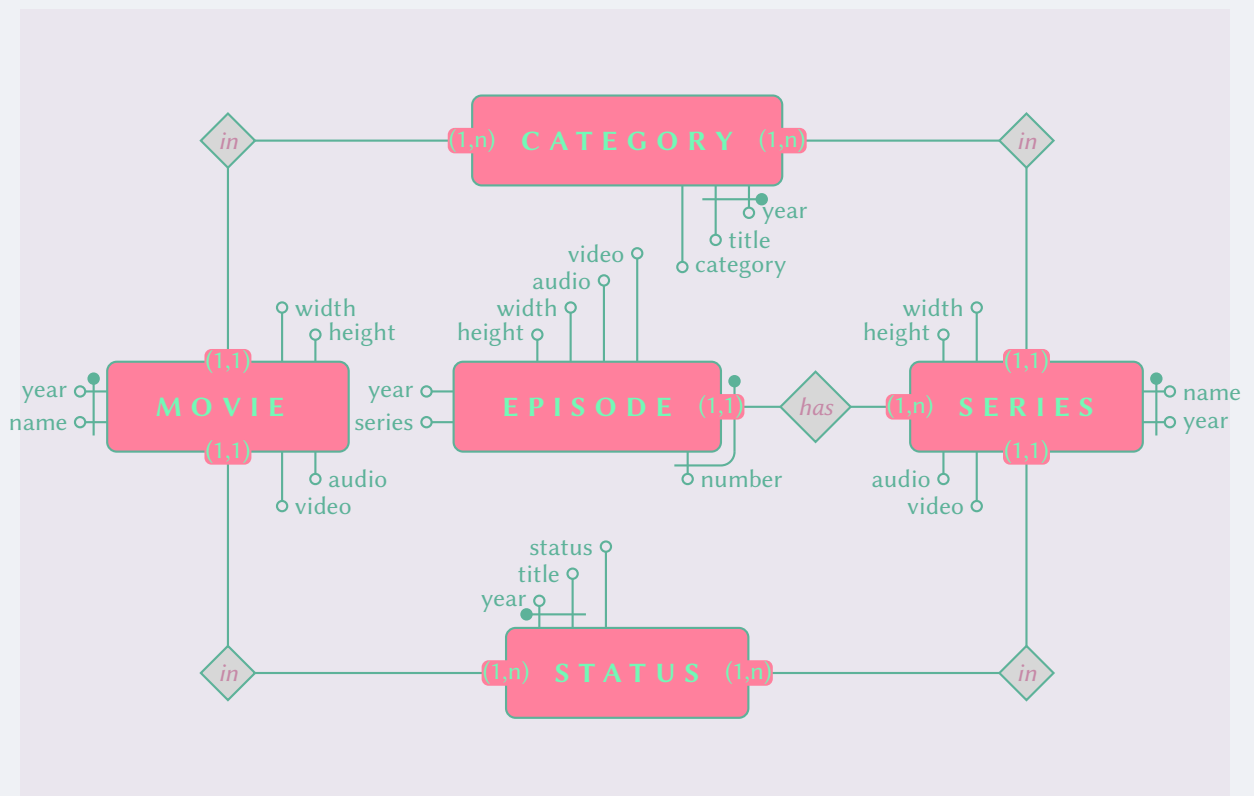


Figure 10: vaporvawe theme (font: Libertinus Sans).

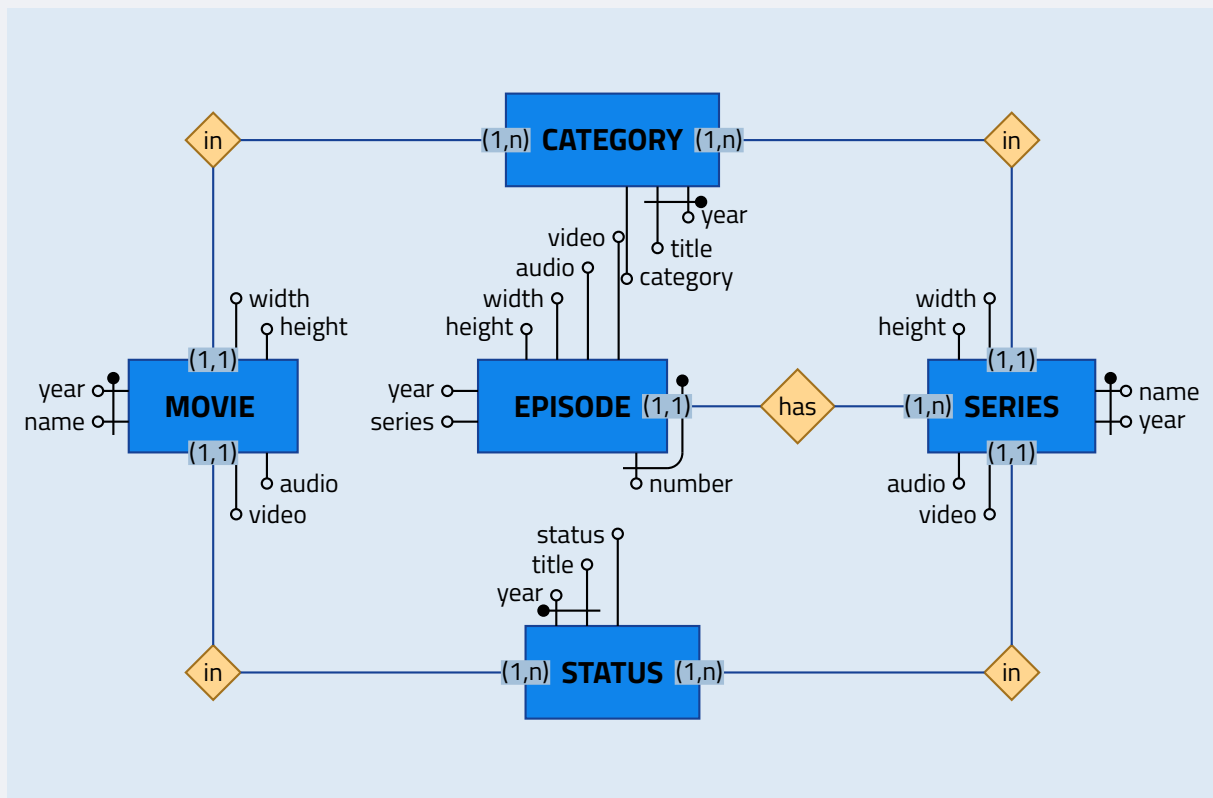


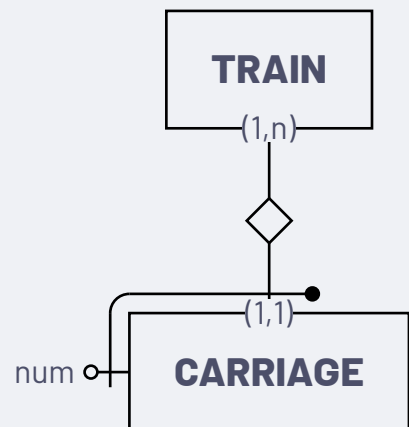
Figure 11: gal1 theme (font: Titillium Web).

4. Advanced usage

4.1. Weak entities

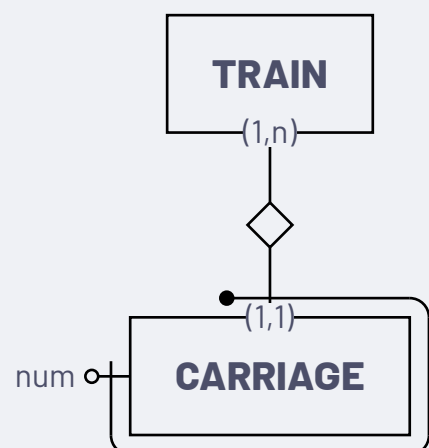
Sometimes there is the need to express a weak constraint, such a carriage number *within the context* of a train; in order to do so specify the weak entity argument with the (attribute, strong-entity, direction) format:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "train",  
    label: "train",  
  )  
  entity(  
    (0, -4),  
    name: "carriage",  
    label: "carriage",  
    attributes: (west: ("num", )),  
    weak-entity: ("num", "train")  
  )  
  relation(  
    entities: ("train", "carriage"),  
    cardinality: ("(1,n)", "(1,1)"),  
  )  
})
```



Changing the direction – now CCW, counter clockwise:

```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "train",  
    label: "train",  
  )  
  entity(  
    (0, -4),  
    name: "carriage",  
    label: "carriage",  
    attributes: (west: ("num", )),  
    weak-entity: ("num", "train", "CCW")  
  )  
  relation(  
    entities: ("train", "carriage"),  
    cardinality: ("(1,n)", "(1,1)"),  
  )  
})
```



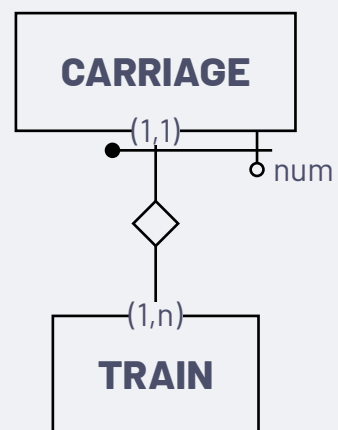
⚠ Warning

By default, you can't relate a weak entity to a strong entity *if the strong entity does not exist yet* – this behaviour mirrors how a possible SQL translation would work:

```
1 create table train(  
2   id int primary key  
3 );  
4 create table carriage(  
5   id int primary key,  
6   foreign key(id) references train(id)  
7 )
```

If you need to, you can specify the location of the strong entity via cardinal points:

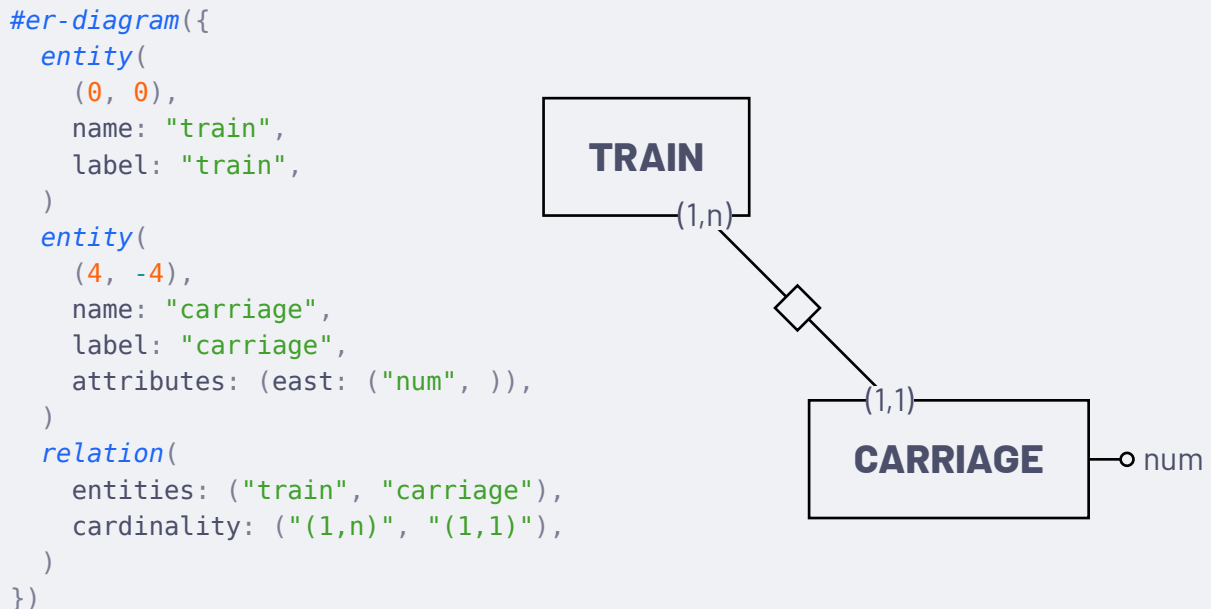
```
#er-diagram({  
  entity(  
    (0, 0),  
    name: "carriage",  
    label: "carriage",  
    attributes: (south: ("num", )),  
    weak-entity: ("num", "south")  
  )  
  entity(  
    (0, -4),  
    name: "train",  
    label: "train",  
  )  
  relation(  
    entities: ("train", "carriage"),  
    cardinality: ("(1,n)", "(1,1)"),  
  )  
})
```



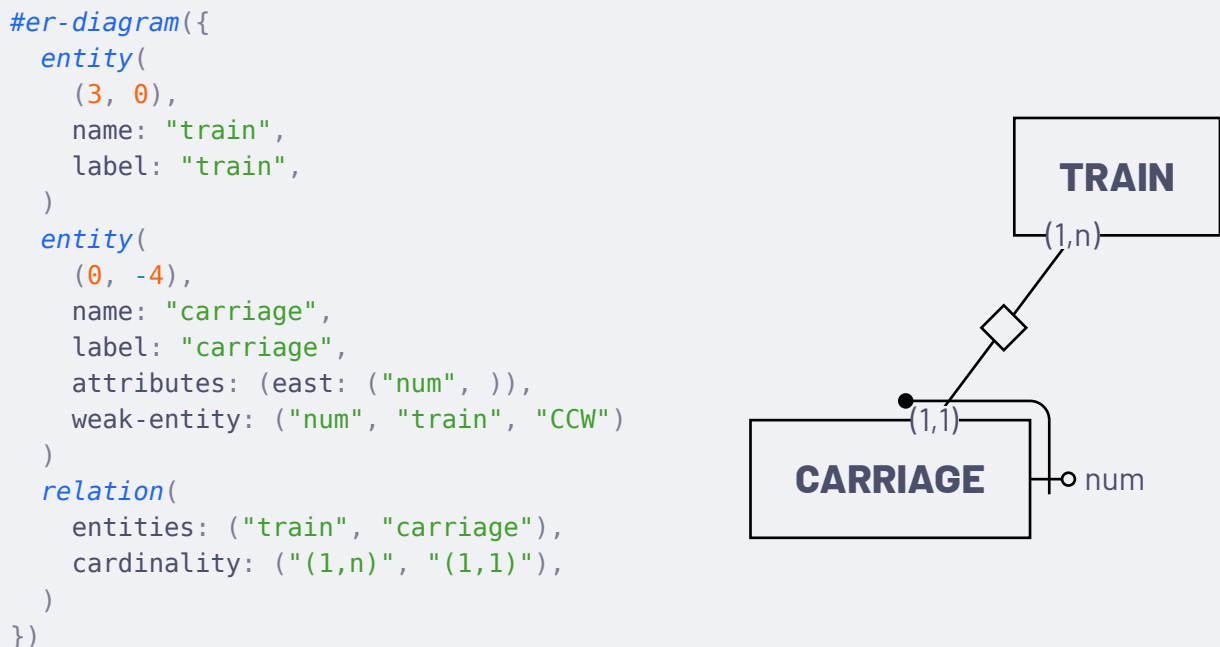
Note carriage and train have been swapped, so that the weak entity has been drawn *before* its corresponding strong entity.

4.2. Unorthodox relation position

Sometimes, due to the density of entities, the relations cannot be placed right above, left, right or below – the package acknowledges this and automatically handles this situation:



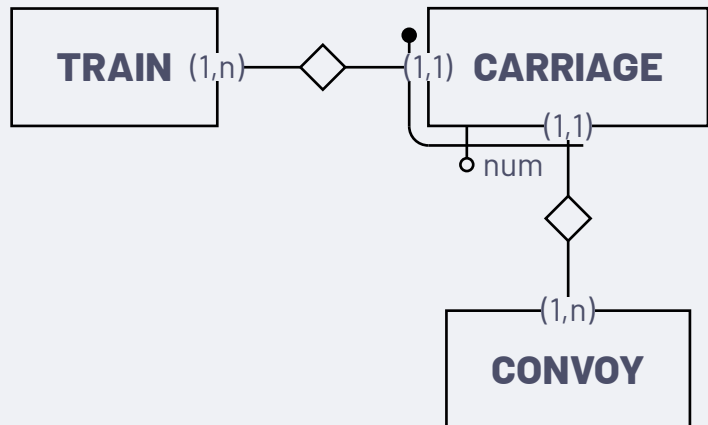
Weak entity It also automatically handles weak entities in such cases:



4.3. Exceptions

For some reason, you might need to create a weak entity that involves two entities but can't – or won't – start from the semantically correct attribute; you can still use the cardinal points:

```
#er-diagram({
  entity(
    (6, 0),
    name: "carriage",
    label: "carriage",
    attributes: (south: ("num", )),
    attributes-position: (south: (alignment: left)),
    weak-entity: ("south", "west")
  )
  entity(
    (0, 0),
    name: "train",
    label: "train",
  )
  entity(
    (6, -4),
    name: "convoy",
    label: "convoy",
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
  relation(
    entities: ("convoy", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



Also do note carriage is the first entity to be drawn.

4.4. Touying integration

Since the package is based on [CeTZ](#), you can pause the animations as you would do directly with CeTZ via [touying-reducer](#):

```
1 #import "@preview/dati-basati:0.1.1" as db
2 #import "@preview/cetz:0.5.2"
3 #import "@preview/touying:0.7.3": *
4
5 #let db-er-diagram = touying-reducer.with(
6   reduce: db.er-diagram,
7   cover: cetz.draw.hide.with(bounds: true),
8 )
9
10 #db-er-diagram({
11   entity(...)
12   (pause,)
13   entity(...)
14 })
```

typ

5. User functions

For *all* the generated documentation, refer to </docs/docs.pdf>.

5.1 dati-basati

Default global theme.

Parameters

```
dati-basati(  
  theme: dictionary ,  
  fill: dictionary ,  
  stroke: dictionary ,  
  radius: dictionary ,  
  spacing: dictionary ,  
  text: dictionary ,  
  attributes-position: dictionary ,  
  misc: dictionary ,  
  body  
) -> dictionary
```

theme dictionary

Global theme for ER diagrams.

Default: (:)

fill dictionary

The fill of each element. Deprecated: use theme.

Default: (:)

stroke dictionary

The stroke of each element. . Deprecated: use theme.

Default: (:)

radius dictionary

The radius of entities, cardinality and hierarchy. . Deprecated: use theme.

Default: (:)

spacing dictionary

The spacing of attributes. . Deprecated: use theme.

Default: (:)

text dictionary

Text formatting of each element. . Deprecated: use theme.

Default: (:)

attributes-position dictionary

The position and direction of attributes. . Deprecated: use theme.

Default: (:)

misc dictionary

Miscellaneous options. . Deprecated: use theme.

Default: (:)

5.2 er-diagram

Draw an Entity-Relation Diagram. This is the function that wraps all the diagrams.

Parameters

```
er-diagram(  
  theme: dictionary ,  
  ..args: dictionary ,  
  body: content  
) -> content
```

theme dictionary

The theme of this diagram.

Default: (:)

..args dictionary

Arguments directly passed to `#cetz.canvas()`.

body `content`

Body of the diagram.

5.3 entity

Draw an entity.

Parameters

```
entity(  
    coordinates: array ,  
    name: str ,  
    label: str content ,  
    attributes: dictionary ,  
    attributes-position: dictionary ,  
    primary-key: str array ,  
    weak-entity: array ,  
    misc: dictionary ,  
    ..args: args  
) -> content
```

coordinates `array`

Coordinates of the entity.

name `str`

Unique name of the entity.

Default: ""

label `str` or `content`

Label for the entity (e.g. Person, Book, Employee...).

Default: "Entity"

attributes `dictionary`

Attributes of the entity (e.g. id, name, surname, address...).

Default: (:)

attributes-position dictionary

Fine tune positioning of attributes.

Default: (:)

primary-key str or array

Primary key(s).

Default: none

weak-entity array

Whether this entity is weak and the corresponding attribute. The format is as follows:

- SQL mirror: (attribute, strong entity, direction)
- With cardinal points: (cardinal pt, cardinal pt, direction)

They can be mixed.

Default: none

misc dictionary

Miscellaneous options:

- weak-entity-intersection

Default: none

..args args

Arguments passed to #std.[rect\(\)](#).

5.4 subentities

Connect subentities to the superentity.

Parameters

```
subentities(  
  hierarchy: "(t,e)" "(p,e)" "(p,o)" ,  
  entity: str,  
  subentities: array  
) -> content
```

hierarchy "(t,e)" or "(p,e)" or "(p,o)"

The hierarchy of the subentities:

- (t,e)=(total, exclusive)
- (p,e)=(partial, exclusive)
- (p,o)=(partial, overlapping)

Default: "(t,e)"

entity str

The superentity, the one to which the subentities will connect.

Default: ""

subentities array

The subentities. Can be just one.

Default: ("",)

5.5 relation

Draw a relation between (sub)entities.

Parameters

```
relation(  
    coordinates: array ,  
    entities: array ,  
    name: str ,  
    label: str ,  
    attributes: dictionary ,  
    cardinality: array ,  
    intersect: bool ,  
    ..args: args  
) -> content
```

coordinates array

The specific coordinates of this relation. If left empty, they will be the middle point between the two entities.

Default: none

entities array

Entities that are tied by this relation.

Default: ()

name str

Unique name of relation.

Default: ""

label str

Label for the relation (e.g. "occupies", "relates", "is part of"...).

Default: ""

attributes dictionary

Attributes of the relation (e.g. id, name, surname, address...).

Default: (:)

cardinality array

Cardinality of the given relation:

Default: ()

intersect bool

Whether to intersect the lines connecting relations instead of a direct connection.

Default: false

..args args

Arguments passed to #std.*rect*().